

# Hyper-Local Economic Network Repair

## Minimum-Viable-Venture + Financial Structuring Engine

SIH26091 | AI-Driven Hyper-Local Business Advisory and Financial Structuring Assistant for Rural Micro-Entrepreneurs

Understand

Map Economy

Find Bottleneck

Synthesize Venture

Prove Resilience

**Proposal identity.** The official problem asks for an AI-driven hyper-local business feasibility and financial-structuring assistant. Our differentiator is to make the decision core explicit: model the local economy as a flow network, identify the structural gap, construct the smallest entrepreneur-feasible venture that repairs it, then test financing and survival before recommending investment.

**How to read this document.** **PS REQUIRED** means directly required by the published SIH26091 problem statement. **OUR PROPOSED CORE** means an algorithmic structure added by this proposal, not an official SIH requirement. **YOU MUST DESIGN/VALIDATE** means the part that must be researched, defined, calibrated, proved or experimentally justified by the team; it must not be copied as if already solved. **SUPPORT** means useful implementation support.

Document Map

- 1

Official problem requirements versus our proposed differentiator
- 2

Proposal architecture, data/evidence model and economic graph
- 3

Economic flow, unmet demand and structural bottleneck detection
- 4

Venture primitives, counterfactual network repair and entrepreneur feasibility
- 5

Minimum Viable Venture algorithm and search structure
- 6

Official PS financial calculator versus our financial optimizer
- 7

Business digital twin, stress testing, minimum regret and staged expansion
- 8

AI boundary, software modules and auditable interfaces
- 9

Exactly what you must invent/interpret/validate
- 10

Proposal preparation, development order, team split and validation
- 11

Professional demo sequence, what not to build and final complete workflow

1. What SIH26091 Actually Requires

**PS REQUIRED** The published statement is not merely “build a chatbot.” It explicitly asks for an NLP-powered multilingual AI business advisory assistant for rural/semi-urban geographies, using basic entrepreneur inputs and producing a localized feasibility study plus a financial roadmap.

1.1 Inputs explicitly requested

| Input                      | Meaning in the prototype                                                                             |
|----------------------------|------------------------------------------------------------------------------------------------------|
| Geographic location        | Village / Block / District; enough to define the hyper-local catchment and competitor/market context |
| Available margin capital   | The entrepreneur’s own available contribution used by the financial calculator                       |
| Proposed business category | Example sector such as dairy, retail, textiles, processing, service, etc.                            |

**YOU MUST DESIGN/VALIDATE** Decide how much extra information is actually needed beyond these minimum inputs: skills, assets, experience, acceptable debt, minimum income, operating time, mobility and risk tolerance. These are proposal-added variables because they improve feasibility, but they should be justified as necessary rather than collected indiscriminately.

1.2 Official Module 1: Hyper-Local Business Feasibility Report

**PS REQUIRED** The required report includes six ideas: market reach in roughly a 5–10 km local context, underserved opportunity, SWOT, local threats, competitor density/mapping, and product/service market value or pricing guidance.

| PS capability        | Weak implementation                 | Our implementation interpretation                                                           |
|----------------------|-------------------------------------|---------------------------------------------------------------------------------------------|
| Market reach         | Count population inside a radius    | Estimate economically reachable demand using roads, travel cost, market access and capacity |
| Opportunity analysis | Ask an LLM for “good opportunities” | Compute residual/unserved demand and structural bottlenecks                                 |
| SWOT                 | Generic generated paragraph         | Generate from measured model outputs and explicit risk factors                              |
| Threats              | Static checklist                    | Supply bottleneck, seasonality, route/supplier dependence and stress scenarios              |
| Competitor mapping   | Count nearby shops                  | Model competitor capacity, accessibility and demand overlap                                 |
| Market value/pricing | Generic price suggestion            | Use observed/local price evidence, delivered cost and demand scenarios with confidence      |

### 1.3 Official Module 2: Smart Financial Calculator and Scheme Router

**PS REQUIRED** The problem statement supplies a specific margin-to-project-cost logic and scheme-routing rules, including loan share, interest/tenure/moratorium examples and an EMI/repayment output. Those rules must be implemented faithfully as a deterministic calculator and verified against the final official scheme documentation before submission/demo.

**Important separation.** The official calculator answers: “Given the prescribed scheme rules, what project cost/loan/repayment is allowed?” Our optimizer answers a different question: “Even if that amount is allowed, what venture scale and financing structure are actually prudent under expected cash flow and risk?” The second part is our proposal-added decision layer.

## 2. Our Proposed Differentiator

**OUR PROPOSED CORE** We reinterpret a business as a change to the local economic network rather than a label selected by AI.

### Central problem

Given a locality, a proposed business sector, entrepreneur constraints and limited capital, find the minimum-cost venture configuration that repairs an important local economic bottleneck while remaining financially resilient.

### 2.1 One intellectual spine

Local evidence → Economic graph → Unserved flow → Bottleneck  
 → Network repair → Minimum viable venture → Financial resilience

**OUR PROPOSED CORE** This graph/repair/MVV/stress-regret chain is not stated by SIH26091. It is the technical differentiator proposed here.

**YOU MUST DESIGN/VALIDATE** Your job is not merely to code these labels. You must define exactly what nodes/edges mean, how demand and capacity are estimated, how bottlenecks are scored, what configurations are searchable, and under what assumptions the algorithms are valid. Those definitions are the real algorithmic research work.

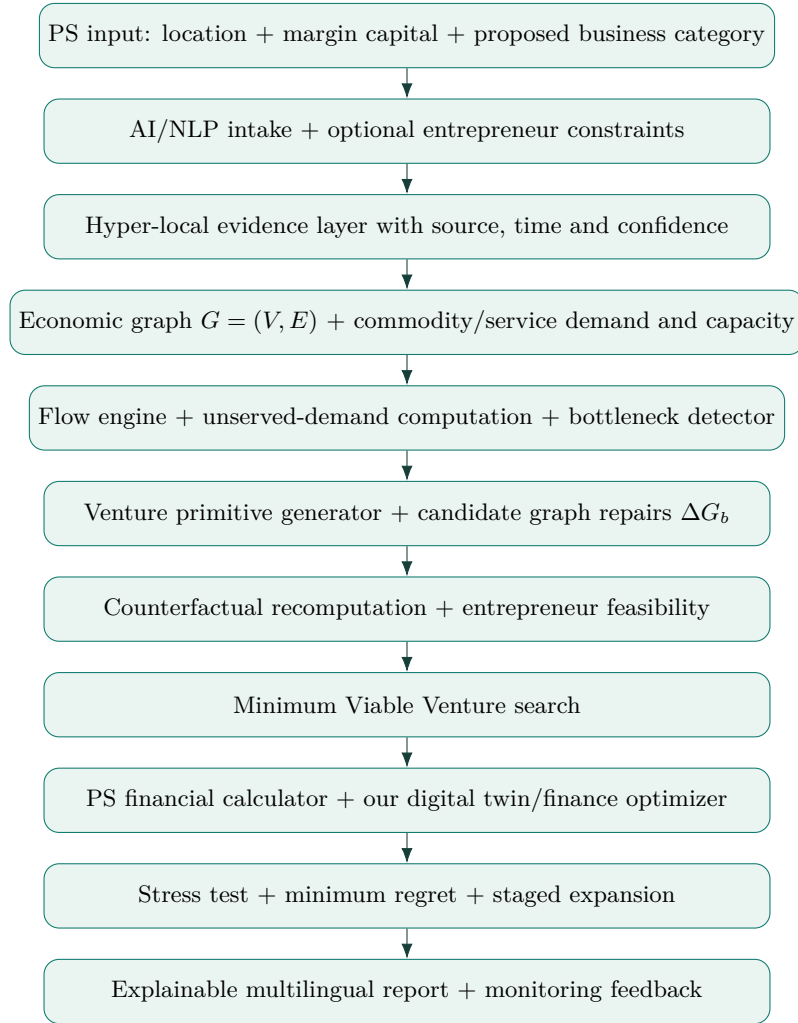
### 2.2 Why this remains faithful to the PS

**PS REQUIRED** SIH26091 already requires hyper-local opportunity analysis, competitor context, market reach, threats and financial structuring. The graph/flow engine is therefore not a different problem; it is our internal method for producing those required outputs from one consistent model. The official business category supplied by the entrepreneur can constrain the primitive library so the system analyzes the requested sector rather than wandering into unrelated ideas.

### 2.3 What this proposal does not claim yet

**YOU MUST DESIGN/VALIDATE** We are not claiming that the flow graph is already a proven representation of every rural economy, that the bottleneck score is universally optimal, or that the MVV search is already exact at large scale. Those become defensible only after the team fixes the model, implements exact tiny-case benchmarks, searches for counterexamples and reports approximation/heuristic boundaries honestly.

### 3. Complete System Architecture



#### 3.1 Decision object

```

VentureDecision = {
  locality, proposedCategory, evidenceSummary,
  detectedBottleneck, ventureConfiguration, startingScale,
  officialSchemeCalculation, financingStructure,
  baseCaseCash, stressCaseCash, failureBoundary,
  rejectedAlternatives, expansionTriggers, confidence, reasons
}

```

**SUPPORT** Keeping one structured decision object prevents the LLM, optimizer, calculator and UI from producing inconsistent numbers.

### 4. Data and Evidence Model

#### 4.1 Entrepreneur state

**OUR PROPOSED CORE** Extend the PS inputs into a structured state only where useful:

$$P = (L, C, S, A, E, T, I_{\min}, R),$$

where  $L$  is locality/reach,  $C$  own capital,  $S$  skills,  $A$  assets,  $E$  experience,  $T$  operating-time availability,  $I_{\min}$  required income and  $R$  risk tolerance.

**YOU MUST DESIGN/VALIDATE** Decide which variables are mandatory, optional or inferred. More variables do not automatically make the model better; each must alter feasibility, risk or the optimization result.

#### 4.2 Evidence record

```

Evidence = {variable, value, geography, source,
  timestamp, confidence, units, validityFlags}

```

**OUR PROPOSED CORE** Every local fact should carry provenance because hyper-local data will be incomplete. Useful classes include consumer/demand proxies, producer supply, existing businesses, road/travel access, transport, storage/processing, suppliers, market days, price observations and financing rules.

### 4.3 Confidence states

Use states such as observed, estimated, stale, unavailable and conflicting. A low-confidence demand estimate should widen stress ranges; it should not be displayed as a precise fact.

**YOU MUST DESIGN/VALIDATE** Define the confidence scale and what it changes mathematically. For example, confidence may control scenario width, trigger additional data collection, or prevent a recommendation if uncertainty is too high.

### 4.4 Prototype data strategy

**SUPPORT** The prototype can combine public datasets, maps/business listings, manually collected local observations and synthetic controlled cases. The crucial rule is to label each source type and never claim synthetic values are real market measurements.

## 5. Hyper-Local Economic Graph

**OUR PROPOSED CORE** Model the locality as

$$G = (V, E).$$

Possible node types: producer clusters, customer clusters, businesses, weekly markets, storage, processors, suppliers and transport/collection points. An edge  $e = (u, v)$  can carry

$$(c_e, k_e, \tau_e, r_e),$$

representing capacity, monetary/logistics cost, travel time and reliability.

### 5.1 Why this is more than competitor mapping

A competitor 3 km away may have tiny capacity or poor access; another 7 km away on a reliable route may serve the same customers efficiently. The graph therefore represents economic accessibility, not only distance.

### 5.2 Product/service layers

For product/service class  $q$ , define local demand  $D_q(v)$  and available supply/capacity  $S_q(v)$ . Depending on the chosen formulation, the graph may be single-commodity per evaluation or multi-commodity across related goods/services.

**YOU MUST DESIGN/VALIDATE** This is one of your main interpretation points. You must choose a tractable graph abstraction. Do not claim a general multi-commodity flow solution if the prototype actually uses a simpler capacitated assignment. State the exact model you implement.

### 5.3 Dynamic update

Prices, travel conditions, market-day access, supplier reliability and observed demand change. Store graph changes locally so only the affected subproblem needs recomputation when possible.

## 6. Economic Flow and Unserved Demand

**OUR PROPOSED CORE** For each class  $q$ , compute how much demand is actually served through the current network. Define

$$U_q(G) = D_q - \text{Served}_q(G).$$

The flow engine may use min-cost flow, capacitated assignment, transportation optimization or another explicitly justified formulation.

### 6.1 Four quantities to keep separate

- **Nominal demand:** estimated desire/need in the catchment.
- **Reachable supply:** supply connected to the area by feasible routes.
- **Effective served demand:** amount that can actually be delivered under capacity/cost constraints.
- **Residual unserved demand:** demand remaining after the current network is used.

**YOU MUST DESIGN/VALIDATE** The hardest part is not running a flow algorithm; it is defining realistic demand and capacity from evidence. Your proposal should explicitly say which quantities are measured, estimated, inferred or scenario-tested.

## 6.2 PS connection

**PS REQUIRED** This engine is our mechanism for producing the official opportunity analysis, competitor context, market reach and threat insights. The PS asks for those outputs; it does not prescribe this graph/flow method.

## 7. Structural Bottleneck Detection

**OUR PROPOSED CORE** A large  $U_q(G)$  says there is an unmet gap; the next question is why it exists. Candidate bottlenecks include:

- insufficient retail/service capacity;
- missing aggregation between many small producers and a market;
- transport edge with low capacity or high delivered cost;
- missing storage/processing causing spoilage or low-value sales;
- single-supplier dependence or route fragility;
- mismatch between demand location and current supply location.

### 7.1 Bottleneck score

A prototype score may combine marginal flow gain obtainable by relaxing a constraint, affected demand, repair cost and confidence. One generic form is

$$B(e) = \frac{\Delta \text{Served}(e)}{\text{RepairCost}(e) + \varepsilon} \cdot \text{Confidence}(e).$$

**YOU MUST DESIGN/VALIDATE** This formula is only a proposal framework, not a finished theorem. You must decide whether the implemented bottleneck is found through residual-network/min-cut structure, sensitivity analysis, marginal capacity tests or a custom heuristic. Then benchmark it against controlled cases where the true bottleneck is known.

### 7.2 Important non-obvious cases

Zero competitors does not imply opportunity if demand is negligible. Many competitors do not imply saturation if capacity is insufficient. A missing connector such as transport may be a stronger opportunity than another shop.

## 8. Venture Primitives and Candidate Network Repairs

**OUR PROPOSED CORE** Do not restrict the engine to a fixed ranking of business names. Represent a venture as a combination of primitives:

{retail, aggregation, transport, storage, processing, delivery, repair, distribution}.

A candidate  $b$  creates a graph modification  $\Delta G_b$ :

$$G \longrightarrow G_b = G + \Delta G_b.$$

### 8.1 Examples

- a transport service adds route capacity between producer and market nodes;
- storage inserts a capacity-preserving node between production and later sale;
- a processor consumes one commodity and creates another output commodity/value stream;
- aggregation connects several small suppliers to one downstream buyer;
- a retail/service outlet adds local service capacity to a customer cluster.

**YOU MUST DESIGN/VALIDATE** Define a small but expressive primitive library for the chosen demo categories. The algorithm cannot synthesize a business unless the cost, capacity, required skills/assets and financial effects of each primitive are encoded.

## 9. Counterfactual Network Recalculation

**OUR PROPOSED CORE** Every candidate must be evaluated by recomputing the local economy after insertion:

$$\text{Benefit}(b) = \text{Served}(G_b) - \text{Served}(G).$$

This is the key difference from an opaque “AI opportunity score.” The system tests what the candidate changes.

### 9.1 Demand redistribution and cannibalization

A new venture can take customers from existing businesses rather than create entirely new served demand. The counterfactual engine should therefore reallocate demand under the same accessibility/capacity rules instead of adding projected revenue independently.

## 9.2 Entrepreneur feasibility

Reject candidates violating available capital, skills/assets, geographic reach, time or debt limits. Economically useful is not automatically entrepreneur-feasible.

**YOU MUST DESIGN/VALIDATE** Decide the feasibility rules and whether they are hard constraints or penalties. Hard legal/physical limits should be constraints; preferences can be scored.

## 10. Minimum Viable Venture - Main Algorithm

**OUR PROPOSED CORE** The central optimization is not “choose the most profitable full-size business.” Search for the smallest venture configuration  $x$  that captures enough local opportunity to be viable:

$$\min_x \text{Investment}(x)$$

subject to

$$\begin{aligned} \text{EconomicBenefit}(x) &\geq B_{\min}, & \text{Income}(x) &\geq I_{\min}, \\ \text{Cash}_t(x) &\geq 0, & \text{OwnCapitalRequired}(x) &\leq C. \end{aligned}$$

### 10.1 Configuration variables

A configuration may include number/capacity of assets, rent-versus-buy choice, route count, operating days, inventory level, staffing, service radius, initial product subset and working-capital buffer.

### 10.2 Search strategy

For tiny cases, enumerate all feasible configurations to create an exact oracle. For larger cases use dominance pruning, branch-and-bound/DP, integer programming, carefully justified greedy structure, or a custom search exploiting monotonicity/locality.

**YOU MUST DESIGN/VALIDATE** This is the strongest place for your own algorithmic contribution. Investigate properties such as: if scale increases, which constraints are monotone? Can one configuration dominate another in cost, capacity and cash safety? Can lower bounds prune whole subtrees? Can the network benefit be cached when only one primitive changes? Do not choose an optimization method before studying the structure.

### 10.3 Example of the desired output

Instead of “start a transport business,” output a configuration such as: rent one vehicle on two market days, serve three producer clusters and one market, maintain a defined working-capital buffer, then buy an asset only after utilization and cash triggers are satisfied.

## 11. Official Financial Calculator Versus Our Financial Optimizer

### 11.1 The PS-required deterministic calculator

**PS REQUIRED** Implement the margin-capital to project-cost/loan calculation, scheme routing, interest/tenure/moratorium logic and repayment schedule exactly as specified by SIH26091. This part should be deterministic, auditable and unit-tested; do not ask an LLM to perform the arithmetic.

The problem statement provides example thresholds/rates/tenures. Before the final SIH submission and live demo, verify the current official scheme rules and preserve the version/date used. The algorithm document should distinguish “PS-provided example logic” from “currently verified scheme rule.”

### 11.2 Our additional financial decision layer

**OUR PROPOSED CORE** Eligibility to borrow does not prove that borrowing the maximum amount is prudent. Let

$$z = (E, L, W, S)$$

represent owner equity, term loan, working-capital facility and eligible support/subsidy. Search for a structure that keeps the chosen venture solvent rather than simply maximizing debt.

A generic resilience formulation is

$$\min_{x,z} \text{Risk}(x, z) + \lambda \text{CapitalBurden}(x, z)$$

subject to minimum income/economic benefit, cash constraints and all scheme rules.

## 11.3 Financial quantities

Track at least project cost, owner margin, debt drawdown, interest, moratorium effect, repayment schedule, fixed/variable operating cost, working capital, cash reserve and entrepreneur income draw.

**YOU MUST DESIGN/VALIDATE** Define what “financially safe” means. Possible measures include non-negative monthly cash, debt-service coverage, minimum reserve, maximum affordable payment and probability/number of stress scenarios survived. Do not use arbitrary thresholds without rationale.

## 12. Business Digital Twin

**OUR PROPOSED CORE** Simulate the proposed venture month by month before recommending it. One accounting backbone is

$$Cash_{t+1} = Cash_t + Revenue_t - Opex_t - DebtService_t - Capex_t - \Delta WC_t.$$

### 12.1 State variables

Demand, units sold, price, revenue, inventory, fixed cost, variable cost, working capital, debt balance, repayment, cash reserve, utilization and entrepreneur draw.

### 12.2 Why it matters

Two ventures can have similar annual profit but very different month-to-month cash failure risk. The twin exposes working-capital gaps that an EMI calculator cannot see.

**YOU MUST DESIGN/VALIDATE** Decide horizon and time step, how seasonality enters, and which revenue/cost assumptions are evidence-based versus scenarios. For a hackathon prototype, a monthly horizon of 24–36 months is easier to explain than an unnecessarily complex continuous model.

## 13. Stress Testing and Failure Boundary

**OUR PROPOSED CORE** Deliberately perturb assumptions: demand down, input cost up, price pressure, supplier delay, route disruption, interest change and seasonal shock. Low-confidence variables should receive wider ranges.

Rather than only reporting “medium risk,” find a failure boundary, for example the smallest demand decline that creates a negative-cash month or violates a repayment safety constraint.

**YOU MUST DESIGN/VALIDATE** Build scenario ranges from evidence or clearly label them as synthetic demonstration assumptions. Stress testing is useful only if the audience knows what is measured and what is hypothetical.

## 14. Minimum Regret and Staged Expansion

### 14.1 Minimum-regret selection

**OUR PROPOSED CORE** When multiple ventures survive, compare them across scenarios  $\Omega$ :

$$\min_b \max_{\omega \in \Omega} \text{Regret}(b, \omega).$$

This protects a low-capital entrepreneur from a high-expected-return choice that fails badly if the forecast is wrong.

**YOU MUST DESIGN/VALIDATE** Define the benchmark inside regret: best outcome in each scenario, survival score, utility, or another metric. If you cannot justify a regret objective, present it as one robust selection option rather than claiming universal superiority.

### 14.2 Staged launch

**OUR PROPOSED CORE** Generate an adaptive path

$$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow \dots$$

instead of forcing the full investment immediately. Example triggers: sustained utilization, minimum cash reserve, proven demand, repayment safety and supplier stability.

**PS REQUIRED** Staged expansion is not explicitly required by SIH26091. It is a proposal-added method for reducing risk while still satisfying the required financial-advisory objective.



## 15. Where AI Is Used - and Where It Stops

| Layer              | AI / ML role                                                                      | Deterministic / algorithmic role                                     |
|--------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------|
| User intake        | Multilingual speech/text understanding and clarification                          | Validate extracted numbers/categories and construct profile          |
| Local evidence     | Optional entity extraction, text classification, uncertain demand/risk estimation | Provenance, geography, confidence, graph construction                |
| Feasibility report | Explain SWOT/threats/opportunity in natural language                              | Flow, bottleneck, competitor/accessibility and price evidence        |
| Financial module   | Explain calculations and scheme meaning                                           | Exact PS calculator, eligibility checks, repayment arithmetic        |
| Decision core      | No unrestricted final choice                                                      | Network repair, MVV, digital twin, robust finance, stress/min-regret |
| Output             | Multilingual explanation and conversational follow-up                             | Freeze exact computed values, evidence and audit trail               |

### Architecture boundary

AI/NLP input → **economic model + algorithms + financial simulation** → AI/NLP explanation.

## 16. Software Modules and Interfaces

| Module                | Responsibility                                                            |
|-----------------------|---------------------------------------------------------------------------|
| Profile/NLP service   | multilingual intake, extraction, clarification                            |
| Evidence service      | source adapters, geography, timestamp, confidence, versioning             |
| Graph builder         | economic nodes/edges, capacities, costs, travel, reliability              |
| Flow engine           | served demand, residual/unserved demand, counterfactual recomputation     |
| Bottleneck engine     | sensitivity/marginal-flow tests and ranked structural gaps                |
| Repair generator      | venture primitives and candidate graph modifications                      |
| MVV optimizer         | minimum viable configuration under entrepreneur/economic constraints      |
| PS finance calculator | exact margin/project-cost/loan/scheme/repayment logic required by PS      |
| Digital twin          | monthly revenue/cost/working-capital/debt/cash simulation                 |
| Robustness engine     | stress scenarios, failure boundary, regret and stage triggers             |
| Explanation/UI        | official feasibility report, comparisons, evidence, multilingual advisory |
| Monitoring service    | post-launch actuals and plan re-evaluation                                |

### 16.1 Core data objects

EntrepreneurProfile, EvidenceRecord, EconomicNode, EconomicEdge, FlowResult, Bottleneck, VenturePrimitive, VentureCandidate, OfficialFinanceResult, FinancialPlan, StressScenario, VentureDecision, MonitoringObservation

## 17. Exactly What You Need to Invent / Interpret / Validate

### YOU MUST DESIGN/VALIDATE

The formulas in this proposal are architecture-level formulations. They are not finished research results. Your main contribution is to turn them into a precise, testable algorithmic model and show where it works, where it fails and why.

| Area           | What this document gives you              | What you must actually do                                                                                                |
|----------------|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Economic graph | Node/edge concept and possible attributes | Choose the exact node types, edge meaning, commodity model, geographic granularity and update rules for the demo         |
| Demand/supply  | Served/unserved-demand concept            | Choose data sources/proxies, units, estimation method and confidence treatment; create controlled cases with known truth |



|                           |                                        |                                                                                                                       |
|---------------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Flow algorithm</b>     | Min-cost flow/assignment as candidates | Select one exact formulation and state assumptions/complexity; do not name multiple algorithms without implementation |
| <b>Bottleneck</b>         | Marginal-flow/repair idea              | Design the score or exact structural criterion; test whether it identifies the true limiting element                  |
| <b>Venture primitives</b> | Primitive library concept              | Encode costs, capacities, skill requirements and graph effects for the chosen business categories                     |
| <b>Counterfactual</b>     | Recompute $G + \Delta G_b$             | Define customer/demand redistribution and prevent double-counting demand/revenue                                      |
| <b>MVV</b>                | Min-investment subject to viability    | Discover search structure, pruning/dominance/lower bounds, exact tiny-instance oracle and larger-instance algorithm   |
| <b>Official finance</b>   | Deterministic PS calculator boundary   | Verify scheme numbers/rules from official source and unit-test calculations                                           |
| <b>Digital twin</b>       | Monthly cash recurrence                | Define revenue, seasonality, working capital, debt, taxes/fees if included, and validation ranges                     |
| <b>Financial safety</b>   | Example cash/DSCR constraints          | Decide which metric and thresholds are justified for the prototype                                                    |
| <b>Stress testing</b>     | Scenario framework                     | Define evidence-based or explicitly synthetic stress ranges and failure-boundary procedure                            |
| <b>Minimum regret</b>     | Robust-selection option                | Define regret/utility and prove or empirically justify why it helps in the chosen scenarios                           |
| <b>Staged expansion</b>   | Trigger-based path                     | Choose measurable triggers and show that staged investment improves downside risk in examples                         |
| <b>AI layer</b>           | Input/output responsibilities          | Choose models, languages, extraction validation and hallucination controls; keep arithmetic outside LLM               |
| <b>Evaluation</b>         | Suggested metrics/baselines            | Build datasets/scenarios, exact oracle for tiny cases, ablations and measured comparisons                             |

## 18. What Is Directly from the PS and What Is Only Our Proposal

| Directly requested by SIH26091                              | Added by this proposal                                              |
|-------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Multilingual NLP/AI business advisory</b>                | Economic flow graph as the internal representation                  |
| <b>Location + margin capital + proposed category inputs</b> | Entrepreneur constraint state beyond minimum inputs                 |
| <b>5–10 km market reach concept</b>                         | Accessibility/capacity-based reach instead of radius-only reach     |
| <b>Opportunity / underserved niches</b>                     | Residual flow and bottleneck detection algorithm                    |
| <b>SWOT and local threats</b>                               | Evidence-derived SWOT + stress/failure-boundary engine              |
| <b>Competitor mapping/density</b>                           | Capacity/accessibility/cannibalization counterfactual model         |
| <b>Pricing / local market value guidance</b>                | Confidence-aware pricing evidence and scenario ranges               |
| <b>Financial structuring calculator</b>                     | Minimum Viable Venture and joint configuration-finance optimization |
| <b>Scheme auto-selection and repayment schedule</b>         | Deterministic rule engine plus additional resilience constraints    |
| <b>Operational/working-capital roadmap</b>                  | Full monthly digital twin                                           |
| <b>Business advisory output</b>                             | Minimum-regret selection, staged expansion and monitoring           |

## 19. How the Proposal Should Be Prepared

**SUPPORT** The SIH proposal/PPT should not begin with equations. First make the official problem fit obvious, then show the algorithmic differentiator and evidence.

### 19.1 Recommended narrative order

1. **Problem:** first-time rural entrepreneurs choose businesses from anecdote and struggle with financial structuring.
2. **Official requirement:** hyper-local feasibility + multilingual AI + smart financial calculator/scheme router.
3. **Gap in obvious solutions:** competitor count, generic SWOT and chatbot advice do not model whether the local economy can actually support the venture.
4. **Our core insight:** treat a venture as a repair to a local economic network.

5. **Algorithm flow:** map economy → flow/unserved demand → bottleneck → candidate repairs → MVV.
6. **Financial proof:** official scheme calculator + digital twin + financing safety + stress boundary.
7. **AI role:** multilingual intake/explanation; deterministic models own numbers and decisions.
8. **Demo:** compare ordinary recommendation versus our counterfactual/MVV output on one locality.
9. **Validation:** exact small-case oracle, baselines, stress scenarios, audit trail, data-source confidence.
10. **Impact:** smaller unnecessary capital, better local fit, transparent repayment obligations, safer staged launch.

## 19.2 What to claim carefully

Do not say “we predict exact local demand” unless validated. Do not say “optimal” unless the algorithm is exact under stated assumptions or the optimality gap is measured. Do not call the graph/repair idea an SIH requirement; call it our proposed differentiator. Do not claim scheme rules are current without verifying the authoritative source used in the final build.

## 20. Development Roadmap - Build in This Order

1. Encode the official PS financial calculator first and unit-test every formula/routing rule.
2. Create a tiny synthetic locality with 10–20 nodes where demand, capacities and true bottleneck are known.
3. Implement one exact baseline flow formulation.
4. Implement unserved demand and bottleneck tests; manually verify results.
5. Define 4–6 venture primitives and their graph deltas.
6. Exhaustively evaluate all tiny candidate repairs to create a ground-truth oracle.
7. Add entrepreneur feasibility constraints.
8. Formalize Minimum Viable Venture search; discover dominance/pruning structure and compare to exhaustive oracle.
9. Build the monthly digital twin and reconcile cash-flow arithmetic on hand-worked examples.
10. Add financing-structure search under PS scheme rules.
11. Add stress scenarios and failure boundary; then minimum-regret/staged expansion only if they improve measured robustness.
12. Add real/local evidence adapters and confidence labels.
13. Add multilingual AI intake/explanation after structured-input core passes tests.
14. Build the dashboard/report and one complete demo story.
15. Add post-launch monitoring only after the pre-launch system is stable.

## 21. Suggested Team Split

|                                 |                                                                                                                          |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Algorithms / mathematics</b> | graph model, flow, bottleneck, network repair, MVV search, pruning, exact oracle, complexity, regret/staging experiments |
| <b>Financial modelling</b>      | PS calculator, scheme rules, digital twin, working capital, debt logic, finance optimizer, stress assumptions            |
| <b>AI / NLP</b>                 | multilingual input, extraction, clarification, explanation, hallucination controls                                       |
| <b>Data / integration</b>       | maps/business/public data, manual surveys, price/supplier evidence, provenance/confidence                                |
| <b>Backend / UI</b>             | APIs, database, audit trail, report/dashboard, comparison views                                                          |
| <b>Validation</b>               | synthetic/local benchmark cases, baselines, user walkthroughs, measured metrics                                          |

Keep the algorithmic engine runnable from deterministic JSON inputs. This lets the core be tested before the LLM or external data sources are ready and prevents integration failures from hiding algorithmic errors.

## 22. Testing and Baselines

### 22.1 Baselines

Compare against: radius-only competitor count; weighted opportunity score; LLM-generated ranking; maximum expected profit; official finance calculator without optimization; exhaustive search on tiny venture configurations; and the proposed MVV/robust system.

### 22.2 Metrics

Measure: unserved demand reduced; flow improvement; starting capital required; owner-margin use; working-capital deficit; break-even; repayment violations; stress scenarios survived; worst-case regret; runtime; memory; optimality gap on tiny cases; and recommendation stability when uncertain inputs change.

## 22.3 Required counterexample-style tests

1. **Zero competitors but no demand:** reject the false opportunity.
2. **Many competitors but unmet capacity:** do not reject only because density is high.
3. **Hidden connector bottleneck:** discover transport/aggregation instead of recommending another retailer.
4. **Best repair too expensive:** synthesize a smaller feasible partial repair.
5. **Base-case profit but cash failure:** restructure finance or reject.
6. **Stress reversal:** safer lower-return option should beat fragile high-return option when robust objective is enabled.
7. **Staged launch:** show a case where gradual investment reduces downside while preserving expansion opportunity.

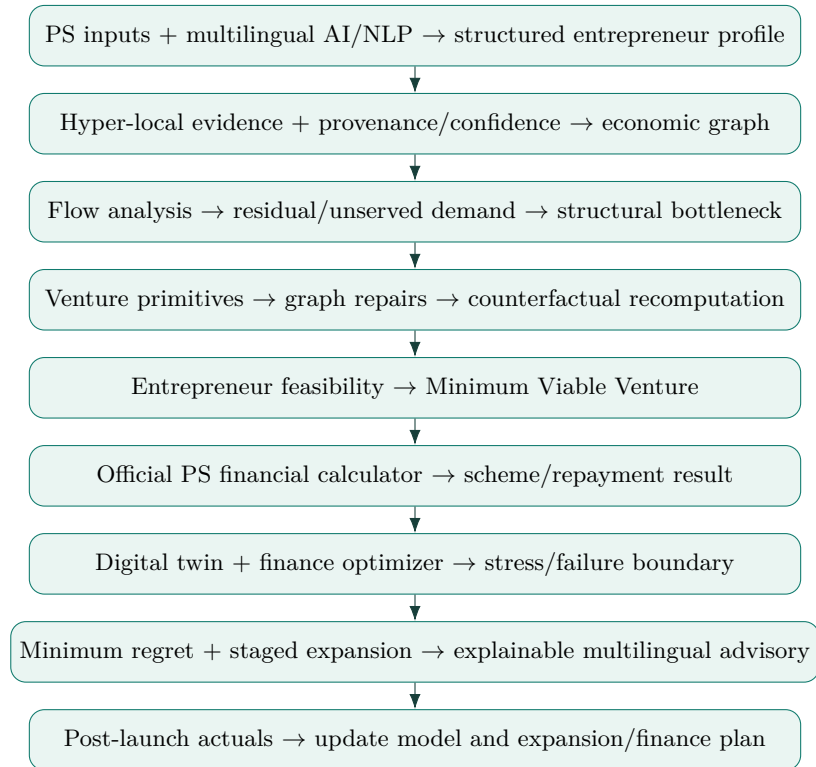
## 23. Professional Demonstration Sequence

1. User speaks/types location, margin capital and proposed business category in a supported language.
2. System validates/structures the inputs and displays the evidence sources used for that locality.
3. Show the ordinary radius-based competitor/market view first.
4. Display the economic graph and current served/unserved flow.
5. Highlight the detected structural bottleneck and why it is limiting demand/supply.
6. Insert two or three candidate ventures and animate the counterfactual flow change.
7. Show why the Minimum Viable Venture is smaller/different from the naive full-scale choice.
8. Run the official PS financial calculator and show project-cost/loan/scheme/repayment output.
9. Run the digital twin and financing optimizer; show working-capital/cash implications.
10. Apply one adverse scenario and display the failure boundary/minimum-regret comparison.
11. Show staged launch triggers and final multilingual explanation with exact evidence/reasons.

## 24. What Not to Build

- chatbot + generic business suggestion as the core;
- “0 competitors = opportunity” logic;
- opaque AI opportunity score with no economic mechanism;
- SWOT generated independently of actual evidence;
- EMI calculator presented as complete financial structuring;
- scheme eligibility decided by LLM prose;
- fabricated hyper-local demand, price or competitor capacity;
- a complex graph with no measurable improvement over simpler baselines;
- claims of exact optimality/novelty without proof or benchmarking.

## 25. Final Complete Workflow



### Final project identity

Do not ask AI to invent a business plan. Build the official SIH26091 feasibility and financial assistant around a transparent economic-network model that detects the missing local structure, synthesizes the smallest entrepreneur-feasible venture that repairs it, and demonstrates financial resilience before recommending investment.

### Final proposal-readiness gates

Before calling the proposal ready, the team should be able to answer all of the following without vague AI language:

1. Which features are directly required by SIH26091, and which are our own additions?
2. What exact graph is implemented, what does an edge mean, and what data populates each parameter?
3. What exact optimization/flow algorithm is running, and under what assumptions is it exact or approximate?
4. Can a tiny instance be solved exhaustively so the MVV result is checked against a known optimum?
5. Can the system explain a non-obvious bottleneck that competitor counting misses?
6. Are the official margin/loan/scheme/repayment calculations independently unit-tested and source-versioned?
7. Does the digital twin reconcile monthly cash, working capital and debt on a hand-worked example?
8. Are stress ranges labelled as measured, sourced, estimated or synthetic?
9. Can the final recommendation trace every number back to evidence or a stated assumption?
10. Can the demo show why our proposed venture is smaller/safer/better structured than at least one simple baseline?

**The proposal is not finished when the diagrams are finished.** It is ready when the team can defend the model definitions, data provenance, algorithmic assumptions, exact-vs-heuristic boundary, financial arithmetic and benchmark results independently of the LLM.